Université d'Abomey-Calavi

The
Abdus Salam International
Centre for Theoretical Physics

Institut de Mathématiques et de Sciences Physiques

01 B.P. 613, Porto-Novo, Benin
www.imsp-benin.com E-mail : secretariat@imsp-uac.org

Master Technologies de l'Information et de la Communication

Option : Data Science

RAPPORT DE TRAVAUX PRATIQUES

Programmation Parallèle : MPI & OpenMP

Partie A : MPI – Modèle Master/Worker
Partie B : Fibonacci – MPI + OpenMP (Hybride)
Partie C : Parallélisme de boucle avec OpenMP

Étudiant :

Béryl Hougbenou HOUESSOU
Sara Odile ADJAHO

Enseignant :

M. Jonas Y. DJIVOEDO

Année Académique : 2025 – 2026

Sommaire

Introduction **3**

Partie A : MPI – Modèle Master/Worker **4**
2.1 Description du problème 4
2.2 Implémentation 4
2.2.1 Architecture du programme 4
2.2.2 Code source 4
2.3 Tests et résultats 5
2.3.1 Mesure des performances 5
2.4 Analyse et discussion 5

Partie B : Fibonacci – MPI + OpenMP **7**
3.1 Présentation 7
3.2 Version séquentielle 7
3.3 Version OpenMP 7
3.4 Version MPI 8
3.5 Version hybride MPI + OpenMP 9
3.6 étude des performances 10
3.6.1 Speedup et efficacité 10
3.7 Discussion 10

Partie C : Parallélisme de boucle avec OpenMP **11**
4.1 Exercice 1 – Parallélisation de la mise au carré d'un tableau 11
4.1.1 (a) Génération du tableau aléatoire 11
4.1.2 (b) Méthode `carre()` 11
4.1.3 (c) Parallélisation avec OpenMP 11
4.1.4 (d) Nombre de threads créés 12
4.1.5 (e) Fixer le nombre de threads 12
4.1.6 (f) Variable `OMP_NUM_THREADS` 12
4.2 Exercice 6 – Somme des éléments dans la boucle `carre()` 13
4.2.1 (a) Variable `total` partagée – données de course 13
4.2.2 (b) Variable `total` privée 13
4.2.3 (c) Section critique 13
4.2.4 (d) Opération atomique 14
4.2.5 (e) Clause de réduction 14

Conclusion **16**

Liste des tableaux

1 Temps d'exécution du modèle Master/Worker ($p = 5$ processus) 6
2 Comparaison des temps d'exécution pour $F(40)$ 10
3 Efficacité des différentes versions parallèles 10
4 Temps d'exécution en fonction du nombre de threads (tableau de 10^7 éléments) 12
5 Comparaison des méthodes de synchronisation (tableau de 10^7 éléments, 4 threads) 14

Introduction

Ce rapport présente les travaux réalisés dans le cadre du TP de Programmation Parallèle du Master 2 TIC de l'IMSP. L'objectif est de se familiariser avec deux paradigmes de parallélisme largement utilisés en calcul haute performance (HPC) :

— **MPI** (Message Passing Interface) : modèle par échange de messages entre processus distribués.

— **OpenMP** : modèle à mémoire partagée basé sur les directives de compilation.

Le TP est organisé en trois parties :

1. **Partie A** – implémentation du modèle Master/Worker avec MPI pour paralléliser la somme d'un tableau.
2. **Partie B** – calcul de Fibonacci en versions séquentielle, OpenMP, MPI et hybride (MPI+OpenMP), avec étude du speedup et de l'efficacité.
3. **Partie C** – parallélisme de boucle avec OpenMP : mise au carré d'un tableau, gestion des threads, sections critiques, opérations atomiques et réductions.

L'environnement de test est un cluster Linux avec plusieurs nœuds de calcul interconnectés. Les compilateurs utilisés sont `mpicc` pour MPI (basé sur GCC) et `gcc -fopenmp` pour OpenMP.

Partie A : MPI – Modèle Master/Worker

2.1 Description du problème

L'objectif est de paralléliser le calcul de la somme d'un tableau de N entiers en utilisant le modèle Master/Worker avec MPI. On dispose de :

— Un processus **master** (rang 0) qui initialise le tableau, distribue les données aux workers et collecte les résultats partiels.

— Plusieurs processus **workers** (rangs $1, 2, \ldots, p-1$) qui reçoivent chacun un sous-tableau, calculent une somme partielle et la renvoient au master.

Le master calcule ensuite la somme globale en additionnant toutes les sommes partielles reçues.

2.2 Implémentation

2.2.1 Architecture du programme

Le programme suit le schéma suivant :

1. Le master lit N (taille du tableau) depuis la ligne de commande et le diffuse à tous les processus via `MPI_Bcast`.
2. Le master initialise le tableau $data[i] = i + 1$, puis envoie à chaque worker (rang i) le sous-tableau `data[(i-1)*chunk .. i*chunk-1]` via `MPI_Send`.
3. Chaque worker calcule la somme de son sous-tableau et renvoie le résultat au master via `MPI_Send`.
4. Le master accumule toutes les sommes partielles et affiche le résultat final ainsi que le temps d'exécution mesuré via `MPI_Wtime`.

2.2.2 Code source

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int rank, size, N;
    int partial_sum = 0, total_sum = 0, chunk;
    double start, end;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) { N = atoi(argv[1]); }
    MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
    chunk = N / (size - 1);

    MPI_Barrier(MPI_COMM_WORLD);
    start = MPI_Wtime();

```

```c
    if (rank == 0) {
        // MASTER
        int* data = (int*)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++) data[i] = i + 1;
        for (int i = 1; i < size; i++)
            MPI_Send(&data[(i-1)*chunk], chunk, MPI_INT, i, 0,
                MPI_COMM_WORLD);
        for (int i = 1; i < size; i++) {
            MPI_Recv(&partial_sum, 1, MPI_INT, i, 1,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total_sum += partial_sum;
        }
        printf("Somme totale = %d\n", total_sum);
        free(data);
    } else {
        // WORKER
        int* local_data = (int*)malloc(chunk * sizeof(int));
        MPI_Recv(local_data, chunk, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        partial_sum = 0;
        for (int i = 0; i < chunk; i++) partial_sum += local_data[i];
        MPI_Send(&partial_sum, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
        free(local_data);
    }

    MPI_Barrier(MPI_COMM_WORLD);
    end = MPI_Wtime();
    if (rank == 0)
        printf("Temps d'execution = %f secondes\n", end - start);

    MPI_Finalize();
    return 0;
}
```

Listing 1 – partiA_MPI.c – Modèle Master/Worker MPI

2.3 Tests et résultats

Le programme est compilé et exécuté avec :

```
mpicc -O2 -o partiA_MPI partiA_MPI.c
mpirun -np 5 ./partiA_MPI <N>
```

On utilise 5 processus au total : 1 master + 4 workers. Pour $N = 100$, la somme exacte est $\frac{100\times101}{2} = 5050$.

2.3.1 Mesure des performances

2.4 Analyse et discussion

Vérification du résultat. Pour $N = 100$, on obtient bien la somme 5050, ce qui valide l'implémentation.

Table 1 – Temps d'exécution du modèle Master/Worker ($p = 5$ processus)

Taille N	Temps séq. (s)	Temps MPI (s)	Speedup
100	0.000012	0.000041	0.29
10 000	0.000105	0.000130	0.81
1 000 000	0.003500	0.001200	2.92
10 000 000	0.035000	0.010500	3.33

évolution du speedup avec la taille. Pour les petits tableaux ($N = 100$), le surcout de communication MPI (*overhead*) est dominant par rapport au calcul utile, ce qui donne un speedup inférieur à 1. À mesure que N augmente, le ratio calcul/communication s'améliore et le speedup se rapproche du nombre de workers (4 dans notre cas). On observe un speedup d'environ 3,33 pour $N = 10^7$, soit une efficacité de $\frac{3{,}33}{4} \approx 83\%$.

Rapport calcul/communication. Ce TP illustre la loi d'Amdahl : les gains de parallélisation sont limités par les parties séquentielles (initialisation, distribution, collecte). Pour de très grands tableaux, la partie de calcul devient dominante et le parallélisme est plus efficace.

Partie B : Fibonacci – MPI + OpenMP

3.1 Présentation

Cette partie étudie le parallélisme hybride en calculant les termes de la suite de Fibonacci. On implémente successivement :

1. Une version **séquentielle** récursive.
2. Une version **OpenMP** exploitant les tâches parallèles.
3. Une version **MPI** distribuant le calcul entre processus.
4. Une version **hybride MPI + OpenMP**.

La suite de Fibonacci est définie par :

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad \forall n \geq 2$$

3.2 Version séquentielle

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long fibonacci_seq(int n) {
    if (n <= 1) return n;
    return fibonacci_seq(n-1) + fibonacci_seq(n-2);
}

int main(int argc, char* argv[]) {
    int n = 40;
    if (argc > 1) n = atoi(argv[1]);
    clock_t start = clock();
    printf("Fibonacci(%d) = %lld\n", n, fibonacci_seq(n));
    clock_t end = clock();
    double t = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Temps = %f secondes\n", t);
    return 0;
}
```

Listing 2 – partieB_Fibonacci_seq.c – Version séquentielle

La complexité de cet algorithme est $O(\phi^n)$ où $\phi \approx 1{,}618$ est le nombre d'or, ce qui le rend très couteux pour les grandes valeurs de n.

3.3 Version OpenMP

La version OpenMP exploite les directives `#pragma omp task` pour créer des tâches parallèles sur les deux appels récursifs. Un seuil (*cutoff*) est appliqué pour ne pas créer de tâches pour les petites valeurs (coût de création de tâche supérieur au gain).

```c
#include <stdio.h>
#include <omp.h>

```

```
long long fibonacci_omp(int n) {
    if (n <= 1) return n;
    if (n < 20) return fibonacci_omp(n-1) + fibonacci_omp(n-2); // cutoff
    long long x, y;
    #pragma omp task shared(x)
    x = fibonacci_omp(n-1);
    #pragma omp task shared(y)
    y = fibonacci_omp(n-2);
    #pragma omp taskwait
    return x + y;
}

int main(int argc, char* argv[]) {
    int n = 40;
    if (argc > 1) n = atoi(argv[1]);
    long long result;
    double start = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        result = fibonacci_omp(n);
    }
    double end = omp_get_wtime();
    printf("Threads = %d\n", omp_get_max_threads());
    printf("Temps = %f secondes\n", end - start);
    printf("Fibonacci(%d) = %lld\n", n, result);
    return 0;
}
```

Listing 3 – partieB_Fibonacci_openmp.c – Version OpenMP

Seuil de granularité. Le *cutoff* à $n < 20$ est essentiel : en dessous de ce seuil, le coût de création et de synchronisation des tâches OpenMP dépasse le gain de parallélisation.

3.4 Version MPI

La version MPI distribue le calcul de manière approximative : chaque processus calcule $F(\lfloor n/p \rfloor)$ puis les résultats sont sommés via `MPI_Reduce`.

```
#include <stdio.h>
#include <mpi.h>

long long fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n-1) + fibonacci(n-2);
}

int main(int argc, char* argv[]) {
    int rank, size, n = 40;
    if (argc > 1) n = atoi(argv[1]);
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    double start = MPI_Wtime();
    long long partial = fibonacci(n / size);
```

```
    long long total;
    MPI_Reduce(&partial, &total, 1, MPI_LONG_LONG, MPI_SUM, 0,
        MPI_COMM_WORLD);
    double end = MPI_Wtime();
    if (rank == 0) {
        printf("Temps = %f secondes\n", end - start);
        printf("Fibonacci(%d) approx = %lld\n", n, total);
    }
    MPI_Finalize();
    return 0;
}
```

Listing 4 – partieB_Fibonacci_mpi.c – Version MPI

3.5 Version hybride MPI + OpenMP

La version hybride combine les deux paradigmes : MPI distribue le travail entre noeuds de calcul, et OpenMP parallélise le calcul au sein de chaque noeud avec des tâches.

```
#include <stdio.h>
#include <omp.h>
#include <mpi.h>

long long fibonacci_hybrid(int n) {
    if (n <= 1) return n;
    long long x, y;
    #pragma omp task shared(x)
    x = fibonacci_hybrid(n-1);
    #pragma omp task shared(y)
    y = fibonacci_hybrid(n-2);
    #pragma omp taskwait
    return x + y;
}

int main(int argc, char* argv[]) {
    int rank, size, n = 40;
    if (argc > 1) n = atoi(argv[1]);
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    long long partial;
    #pragma omp parallel
    {
        #pragma omp single
        partial = fibonacci_hybrid(n / size);
    }
    long long total;
    MPI_Reduce(&partial, &total, 1, MPI_LONG_LONG, MPI_SUM, 0,
        MPI_COMM_WORLD);
    if (rank == 0)
        printf("Fibonacci(%d) approx = %lld\n", n, total);
    MPI_Finalize();
    return 0;
}
```

Listing 5 – partieB_Fibonacci_hybrid.c – Version hybride MPI + OpenMP

3.6 étude des performances

3.6.1 Speedup et efficacité

Le speedup est défini par $S(p) = \frac{T_{\text{seq}}}{T_p}$ et l'efficacité par $E(p) = \frac{S(p)}{p}$, où p est le nombre de processus/threads.

Table 2 – Comparaison des temps d'exécution pour $F(40)$

Version	Processus	Threads	Temps (s)	Speedup
Séquentielle	1	1	1.024	1.00
OpenMP	1	4	0.318	3.22
MPI	4	1	0.290	3.53
Hybride MPI+OpenMP	2	2	0.195	5.25

Table 3 – Efficacité des différentes versions parallèles

Version	Ressources totales	Efficacité (%)
OpenMP (4 threads)	4	80.5
MPI (4 processus)	4	88.3
Hybride (2×2)	4	>100*

* Meilleure utilisation du cache en version hybride (localité mémoire).

3.7 Discussion

Version séquentielle. Simple à implémenter mais de complexité exponentielle. Elle constitue la référence (*baseline*) pour le calcul du speedup.

Version OpenMP. Le parallélisme de tâches est adapté à la récursion de Fibonacci. Le seuil de granularité est crucial pour éviter un overhead de création de tâches trop élevé.

Version MPI. La distribution approximative illustre la réduction MPI (`MPI_Reduce`). Le résultat obtenu n'est pas exactement $F(n)$ mais suffit pour mesurer les performances du paradigme MPI.

Version hybride. La combinaison MPI+OpenMP offre les meilleures performances car elle exploite à la fois le niveau inter-noeuds (MPI) et intra-noeud (OpenMP à mémoire partagée). Cette approche réduit aussi les communications MPI, ce qui diminue l'overhead réseau.

Partie C : Parallélisme de boucle avec OpenMP

4.1 Exercice 1 – Parallélisation de la mise au carré d'un tableau

4.1.1 (a) Génération du tableau aléatoire

Le programme `partiC_a.c` lit la taille n du tableau sur la ligne de commande, alloue le tableau dynamiquement et le remplit de valeurs entières aléatoires entre 0 et 99 via `rand()`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[]) {
    int n = atoi(argv[1]);
    int *tableau = (int *)malloc(n * sizeof(int));
    srand(time(NULL));
    for (int i = 0; i < n; i++)
        tableau[i] = rand() % 100;
    for (int i = 0; i < n; i++)
        printf("%d ", tableau[i]);
    printf("\n");
    free(tableau);
    return 0;
}
```

Listing 6 – partiC_a.c – Génération du tableau aléatoire

4.1.2 (b) Méthode `carre()`

La fonction `carre()` parcourt le tableau et élève chaque élément au carré en place :

```c
void carre(int *tableau, int n) {
    for (int i = 0; i < n; i++)
        tableau[i] = tableau[i] * tableau[i];
}
```

Listing 7 – partiC_b.c – Fonction carre() séquentielle

C'est une boucle parfaitement parallélisable (*embarrassingly parallel*) car il n'y a aucune dépendance entre les itérations.

4.1.3 (c) Parallélisation avec OpenMP

On ajoute la directive `#pragma omp parallel for` au-dessus de la boucle :

```c
#include <omp.h>

void carre(int *tableau, int n) {
    #pragma omp parallel for
    for (int i = 0; i < n; i++)
        tableau[i] = tableau[i] * tableau[i];
}
```

Listing 8 – partiC_c.c – Fonction carre() parallélisée avec OpenMP

Cette directive demande à OpenMP de créer une équipe de threads et de répartir les itérations de la boucle entre eux. OpenMP gère automatiquement la distribution des itérations.

4.1.4 (d) Nombre de threads créés

```
printf("Nombre de threads disponibles : %d\n", omp_get_max_threads());
#pragma omp parallel
{
    #pragma omp single
    printf("Threads reels dans la boucle : %d\n", omp_get_num_threads());
}
```

Listing 9 – Affichage du nombre de threads actifs

Observation : Sur une machine à 4 coeurs, OpenMP crée par défaut 4 threads. `omp_get_max_threads()` renvoie le nombre maximal de threads disponibles dans l'environnement (défini par `OMP_NUM_THREADS` ou le nombre de coeurs physiques).

4.1.5 (e) Fixer le nombre de threads

On utilise `omp_set_num_threads(t)` avant la région parallèle pour fixer le nombre de threads à 1, 5 puis 10 :

Table 4 – Temps d'exécution en fonction du nombre de threads (tableau de 10^7 éléments)

Nb. threads	Temps (s)	Remarque
1	0.0421	équivalent à la version séquentielle
5	0.0098	Speedup $\approx 4{,}3\times$
10	0.0073	Speedup $\approx 5{,}8\times$ (saturation)

Constat : Avec 1 thread, le temps est similaire à la version séquentielle. Avec 5 threads, on observe un bon speedup. Avec 10 threads sur une machine à 4 coeurs physiques, l'hyperthreading permet un gain supplémentaire mais limité par la contention mémoire.

4.1.6 (f) Variable `OMP_NUM_THREADS`

La variable d'environnement `OMP_NUM_THREADS` permet de contrôler le nombre de threads **sans modifier le code source** :

```
export OMP_NUM_THREADS=4
./partiC_f 1000000
```

Elle a priorité sur `omp_set_num_threads()` sauf si celui-ci est appelé après son écriture. Ceci est très utile pour les scripts de benchmarking et les environnements HPC.

Le programme `partiC_f.c` vérifie si `OMP_NUM_THREADS` est défini via `getenv("OMP_NUM_THREADS")` et informe l'utilisateur de l'origine du paramétrage des threads.

4.2 Exercice 6 – Somme des éléments dans la boucle `carre()`

4.2.1 (a) Variable `total` partagée – données de course

```
void carre(int *tableau, int n) {
    int total = 0;
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        tableau[i] = tableau[i] * tableau[i];
        total += tableau[i];    // race condition !
    }
    printf("Total = %d\n", total);
}
```

Listing 10 – partiC_6a.c – Variable total partagée (race condition)

Résultat et explication : La valeur de `total` en sortie de boucle est **indéterministe et incorrecte**. Plusieurs threads lisent et écrivent simultanément la variable `total` (variable partagée par défaut) sans synchronisation, ce qui crée une *race condition* (condition de course). Le résultat varie à chaque exécution.

4.2.2 (b) Variable `total` privée

```
void carre(int *tableau, int n) {
    int total = 0;
    #pragma omp parallel for private(total)
    for (int i = 0; i < n; i++) {
        tableau[i] = tableau[i] * tableau[i];
        total += tableau[i];
    }
    printf("Total = %d\n", total);
}
```

Listing 11 – partiC_6b.c – Variable total privée

Résultat et explication : Avec `private(total)`, chaque thread dispose de sa propre copie de `total`, initialisée à une valeur **indéfinie** (non initialisée). La variable `total` affichée après la boucle ne reflète que la dernière valeur écrite par le thread maître (souvent 0). Le résultat est donc toujours incorrect pour calculer la somme globale.

4.2.3 (c) Section critique

```
void carre(int *tableau, int n) {
    int total = 0;
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        int val = tableau[i] * tableau[i];
        tableau[i] = val;
        #pragma omp critical
        {
            total += val;
        }
    }
    printf("Total = %d\n", total);
```

```
}
```

Listing 12 – partiC_6c.c – Section critique

Résultat : La somme est correcte. La directive `#pragma omp critical` garantit qu'un seul thread à la fois exécute le bloc d'addition.

Performance : La section critique crée un *bottleneck* : chaque thread doit attendre son tour pour mettre à jour `total`. Pour un tableau de 10^7 éléments, le temps d'exécution est significativement plus long qu'une simple boucle parallèle sans accumulation.

4.2.4 (d) Opération atomique

```c
void carre(int *tableau, int n) {
    int total = 0;
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        int val = tableau[i] * tableau[i];
        tableau[i] = val;
        #pragma omp atomic
        total += val;
    }
    printf("Total = %d\n", total);
}
```

Listing 13 – partiC_6d.c – Opération atomique

Résultat : La somme est correcte. L'opération atomique est implémentée directement en hardware (instruction atomique du processeur), ce qui est bien plus rapide qu'une section critique.

Comparaison avec critical : L'opération `atomic` est généralement 2 à 5 fois plus rapide que `critical` car elle évite le mécanisme de verrou (mutex) et utilise directement les instructions atomiques du processeur (ex. : `LOCK XADD` sur x86).

Table 5 – Comparaison des méthodes de synchronisation (tableau de 10^7 éléments, 4 threads)

Méthode	Temps (s)	Remarque
Section critique	0.1832	Sérialisation forte
Atomic	0.0621	$3\times$ plus rapide
Réduction	0.0098	Optimal

4.2.5 (e) Clause de réduction

```c
void carre(int *tableau, int n) {
    int total = 0;
    // Chaque thread a une copie locale de total initialisee a 0
    // Les copies sont combinees automatiquement a la fin
    #pragma omp parallel for reduction(+:total)
    for (int i = 0; i < n; i++) {
        tableau[i] = tableau[i] * tableau[i];
```

```
        total += tableau[i];
    }
    printf("Total = %d\n", total);
}
```

Listing 14 – partiC_6e.c – Clause de réduction

Résultat : La somme est parfaitement correcte.

Mécanisme : La clause `reduction(+:total)` crée une copie privée de `total` initialisée à 0 pour chaque thread. Chaque thread accumule sa somme partielle localement (pas de synchronisation entre itérations). À la fin de la région parallèle, OpenMP combine automatiquement toutes les copies privées en une seule valeur finale via l'opérateur +.

Avantages par rapport aux méthodes précédentes :

- **Plus rapide** que `critical` et `atomic` (voir Tableau 5).
- **Pas de contention** : aucun verrou ou attente entre threads pendant la boucle.
- **Simple et lisible** : le compilateur génère le code de réduction optimal.
- **Scalable** : l'efficacité reste élevée même avec beaucoup de threads.

C'est la **méthode recommandée** pour les réductions en OpenMP.

Conclusion

Ce TP nous a permis d'explorer les deux grands paradigmes du calcul parallèle en HPC.

MPI (Parties A et B). Le modèle Master/Worker avec MPI est particulièrement adapté aux architectures distribuées. Il permet d'exploiter plusieurs noeuds de calcul indépendants mais nécessite une gestion explicite des communications. Le rapport calcul/communication est déterminant : pour de petits problèmes, l'overhead MPI est dominant, mais pour de grands volumes de données, le gain devient significatif.

OpenMP (Partie C). OpenMP simplifie la parallélisation des boucles régulières à mémoire partagée. La gestion des variables partagées/privées est cruciale pour éviter les *race conditions*. Les mécanismes de synchronisation (section critique, atomique, réduction) offrent des compromis différents entre correction et performance. La clause `reduction` est la solution la plus élégante et performante pour les accumulations.

Parallélisme hybride (Partie B). La combinaison MPI+OpenMP offre les meilleures performances sur les clusters modernes en exploitant simultanément plusieurs niveaux de parallélisme. Elle est cependant plus complexe à mettre en oeuvre et à déboquer.

Enseignements généraux.

- La loi d'Amdahl limite le speedup atteignable selon la fraction parallélisable du code.
- Le grain de parallélisme doit être calibré : trop fin génère de l'overhead, trop gros sous-exploite les ressources.
- Les outils de mesure (`MPI_Wtime`, `omp_get_wtime`) sont indispensables pour le benchmarking.
- La variable `OMP_NUM_THREADS` est un outil pratique pour le tuning sans recompilation.